

Logical Operators in Java

Logical operators are used to perform logical operations on boolean expressions. They help combine two or more conditions to produce a single boolean result (`true` or `false`). These operators are typically used in control statements like `if`, `while`, and `for`.

1. List of Logical Operators

Operator	Description	Example
<code>&&</code>	Logical AND: Returns <code>true</code> if both operands are true	<code>a > 10 && b < 20</code>
<code>,</code>		<code>,</code>
<code>!</code>	Logical NOT: Negates the boolean value	<code>!(a > 10)</code>

2. Description of Logical Operators

1. Logical AND (`&&`)

- Returns `true` if **both** conditions are `true`.
- If any condition is `false`, the result is `false`.

Syntax:

```
condition1 && condition2
```

Example:

```
public class LogicalAndExample {
    public static void main(String[] args) {
        int a = 15, b = 25;

        if (a > 10 && b < 30) {
            System.out.println("Both conditions are true.");
        } else {
            System.out.println("At least one condition is false.");
        }
    }
}
```

```
}  
}
```

2. Logical OR (||)

- Returns `true` if **at least one** condition is `true`.
- If all conditions are `false`, the result is `false`.

Syntax:

```
condition1 || condition2
```

Example:

```
public class LogicalOrExample {  
    public static void main(String[] args) {  
        int a = 5, b = 35;  
  
        if (a < 10 || b < 30) {  
            System.out.println("At least one condition is true.");  
        } else {  
            System.out.println("Both conditions are false.");  
        }  
    }  
}
```

3. Logical NOT (!)

- Reverses the boolean value of the condition.
- If the condition is `true`, the result is `false`, and vice versa.

Syntax:

```
!condition
```

Example:

```
public class LogicalNotExample {  
    public static void main(String[] args) {  
        boolean isRaining = true;
```

```
        if (!isRaining) {
            System.out.println("It's not raining. You can go outside.");
        } else {
            System.out.println("It's raining. Stay indoors.");
        }
    }
}
```

3. Truth Table for Logical Operators

Logical AND (&&)

Condition 1	Condition 2	Result
true	true	true
true	false	false
false	true	false
false	false	false

Logical OR (||)

Condition 1	Condition 2	Result
true	true	true
true	false	true
false	true	true
false	false	false

Logical NOT (!)

Condition	Result
true	false
false	true

4. Comprehensive Example

```
public class LogicalOperatorsDemo {
    public static void main(String[] args) {
        int a = 15, b = 25, c = 5;

        // Logical AND (&&)
        if (a > 10 && b > 20) {
            System.out.println("Both conditions are true.");
        } else {
            System.out.println("At least one condition is false.");
        }

        // Logical OR (||)
        if (a < 10 || c < 10) {
            System.out.println("At least one condition is true.");
        } else {
            System.out.println("Both conditions are false.");
        }

        // Logical NOT (!)
        boolean isSunny = false;
        if (!isSunny) {
            System.out.println("It's not sunny outside.");
        } else {
            System.out.println("It's sunny outside.");
        }
    }
}
```

5. Output

```
Both conditions are true.
At least one condition is true.
It's not sunny outside.
```

6. Key Points

1. Short-Circuiting:

- In `&&`, the second condition is not evaluated if the first condition is `false`.

- In `||`, the second condition is not evaluated if the first condition is `true`.
 - This behavior improves performance by avoiding unnecessary evaluations.
2. Logical operators work only with boolean values and expressions.
 3. Combining relational and logical operators is common in complex conditions.
-

7. Common Mistakes

1. Using `&` instead of `&&` or `|` instead of `||`:

- `&` and `|` are bitwise operators, not logical operators. They evaluate both conditions, even if the result can be determined from the first condition.

```
if (a > 10 & b < 20) { // Incorrect usage
}
```

2. Parentheses for Clarity:

- Use parentheses in complex expressions to avoid confusion and improve readability.

```
if ((a > 10 && b < 20) || (c == 5)) {
    // Logical grouping
}
```